

DEPARTMENT OF COMPUTER SCIENCE

TDT4186 - OPERATING SYSTEMS

Exercise 1 (Optional)

C-Programming

Introduction

Hello, and welcome to the first exercise sheet. Below are theoretical and practical questions to help you assess what you understand and identify any areas that may be unclear to you. We will release a reference solution for practical and theoretical exercises in two weeks.

1 Theory

For the theoretical exercises, you can find some of the answers in the lecture and exercise lecture material. Some questions also require you to integrate the material taught and develop new ideas based on what you have learned.

1. What do you have to do before you can run your C program? In other words, which steps do you have to repeat if you change anything in your code so that the code produces a new output?
2. As we highlighted in the exercise lecture, the compiler is your friend. With what can the compiler help you? Which things can the compiler not help you with? Which other tools can you use if the compiler cannot provide support?
3. Scenario: After you start your program, it runs for a few seconds before it crashes. The only output to the console immediately before the crash is `SEGFAULT`. Which tools could be useful to debug such an error?
4. **Pointer arithmetic** : A pointer is a special type of variable that contains an address that points to a memory location. As with any normal variable, you can apply operations to the value stored in a pointer variable, but it has some minor differences in behaviour.

For this exercise, read the code block below carefully. If you want to read up on pointers and how to operate on them, you can find a chapter on pointers on Blackboard¹.

```
1  struct point
2  {
3      int x;
4      int y;
5      int z;
6  };
7
8  int main()
9  {
10     struct point pArray[4] = {{0, 0, 0}, {0, 0, 0}, {0, 0, 0}, {0, 0, 0}};
11     pArray[1].z = 1025;
12     int *i = (int *) (pArray + 1);
13     *i = 4;
14     i = (int *) (((void *) i) + 9);
15     char *c = (char *) i;
16     printf("c: %d\n", *c);
17     *((int *) (c + 4) + 1) = 5;
18     printf("%d\n", pArray[2].y);
19 }
```

- a) Which values does `pArray` store after line 10 was executed? What values would it be without the part after the `=`?
- b) On line 13, where is the 4 stored? Is this a valid storage location, or are we touching storage that has not been allocated?
- c) What will be printed on line 16?

¹https://ntnu.blackboard.com/ultra/courses/_38311_1/cl/outline

-
- d) What will be printed on line 18?
- e) What would be the result of the following snippet? Explain your solution. Would it be different if we cast `i` to a short?
- Assume 32-bit integers (=4 bytes).

```
1  int *i = 0;
2  unsigned long diff = (unsigned long)(i + 2) - (unsigned long)i;
3  printf("%lx\n", diff);
```

2 Practical Exercises

To complete the practical exercises, you will need a working C compiler. You will find a VirtualBox image containing a Linux installation with the required tools preinstalled in the **Exercise Tools** folder on BlackBoard. Alternatively, you can install the necessary tools directly on your machine. Exercise 1 below guides you through both processes.

1. For the first exercise this week, either download the VirtualBox image and install VirtualBox (or UTM for Mac Users) on your laptop or follow the guide on Blackboard that details the installation of the C development environment.

If you are using the provided VirtualBox image, you can use the following login information:

Username	lab
Password	lab

Once the installation is finished, you can test if the setup works by trying out the following quick exercises:

- (a) Look at the `hello_world.c` file from the handout code. Use `GCC` to produce an executable. Try executing the binary. What is the output of the program?
 - (b) Compile the file `crash.c`. When you try to execute the file, you will see it output the message "SEGFAULT". Please review the program to determine what it should output. Use the installed debugging tools (GDB and Valgrind) to debug the program. Can you fix the program?
2. Write a program that takes a path to a directory as an optional argument. If no argument is provided, figure out the current working directory. List the directory's content in alphabetical order and print it out to `stdout`. If you cannot list the directory due to missing permissions, the provided path points to a file, or the path does not exist, output an appropriate error message to `stderr`.

3 Deliverables

Please deliver a `.zip` file with the following contents containing your changes to the code:

- A document with the answers to the theory questions.
- `crash.c`